

Java LinkedList — Practice Questions & Answers

Common exam / interview style questions with full answers.

Q1. What is a LinkedList, and how is it different from an array?

A LinkedList stores elements as separate Node objects connected by references (next/prev), rather than in one continuous block of memory like an array. It grows and shrinks by re-linking nodes instead of resizing or shifting, but it loses direct indexed access — reaching element *i* means walking *i* links from an end.

Q2. What does a Node in Java's LinkedList contain?

Three fields: `item` (the stored value), `next` (reference to the following node), and `prev` (reference to the preceding node). The list object itself only tracks two references, `first` and `last`.

Q3. Is Java's LinkedList singly or doubly linked?

Doubly linked. Every node holds both a `next` and a `prev` reference, so the list can be traversed in either direction and both ends support $O(1)$ insert/remove.

Q4. What interfaces does LinkedList implement, and why does that matter?

It implements both `List` and `Deque`. Implementing `List` gives it `add/get/remove/sort` like `ArrayList`. Implementing `Deque` means the same object can also behave as a stack (`push/pop`) or a queue (`offer/poll`) without needing a separate class.

Q5. Compare the time complexity of `get(i)` on ArrayList vs LinkedList.

`ArrayList`: $O(1)$, because it is array-backed and can jump straight to the memory address of index *i*. `LinkedList`: $O(n)$, because there is no direct indexing — it must walk from `first` or `last`, one link at a time, until it reaches index *i*.

Q6. Why is inserting at the front $O(1)$ for LinkedList but $O(n)$ for ArrayList?

`LinkedList`'s `addFirst(x)` only creates one new node and re-points two references (the new node's `next`, and the old first node's `prev`). `ArrayList`'s `add(0, x)` must shift every existing element one position to the right to make room at index 0, which is $O(n)$.

Q7. When would you prefer LinkedList over ArrayList?

When the program frequently inserts or removes elements at the start/end (or via an iterator at a known position), and rarely needs random indexed access — for example, implementing a stack, a queue, an undo history, or a playlist where songs are added/removed constantly but rarely accessed by position.

Q8. When would you prefer ArrayList over LinkedList?

When the program reads elements by index frequently (e.g. `list.get(i)` in tight loops), since `ArrayList` gives $O(1)$ access versus `LinkedList`'s $O(n)$. `ArrayList` also has lower memory overhead per element and better CPU cache performance since its data is contiguous.

Q9. What is wrong with this code, and how would you fix it?

```
for (int i = 0; i < linkedList.size(); i++) {  
    System.out.println(linkedList.get(i));  
}
```

Each call to `get(i)` re-walks the chain from an end, so the whole loop costs $O(n^2)$ instead of $O(n)$. Fix it by using a for-each loop (`for (int v : linkedList)`) or an iterator, both of which walk the chain exactly once.

Q10. What is the difference between `remove(1)` and `remove(Integer.valueOf(1))` on a `LinkedList<Integer>`?

`remove(1)` passes an `int`, so Java resolves it to the `remove(int index)` overload and removes whatever element sits at index 1. `remove(Integer.valueOf(1))` passes an `Integer` object, so Java resolves it to `remove(Object o)` and removes the first element whose value equals 1.

Q11. How do you use a LinkedList as a stack? Give the two key methods.

`push(x)` adds to the front, and `pop()` removes from the front, giving Last-In-First-Out (LIFO) order. Both are $O(1)$ because they operate at the head of the list.

Q12. How do you use a LinkedList as a queue? Give the two key methods.

`offer(x)` adds to the back, and `poll()` removes from the front, giving First-In-First-Out (FIFO) order. Both are $O(1)$ since

LinkedList tracks both ends.

Q13. Why can't you binary search a LinkedList efficiently, even if it's sorted?

Binary search needs to jump directly to the middle element in $O(1)$, which arrays support via indexing. A LinkedList has no such shortcut — reaching the "middle" node still requires walking through every node before it, so any binary search implementation degrades toward $O(n)$ per lookup, losing the benefit.

Q14. What is the output of the following code?

```
List<Integer> list = new LinkedList<>(List.of(10, 20, 30, 40));
list.removeFirst();
list.removeLast();
list.addFirst(5);
System.out.println(list);
```

[5, 20, 30] . Step by step: start [10,20,30,40] → removeFirst() removes 10 → [20,30,40] → removeLast() removes 40 → [20,30] → addFirst(5) → [5,20,30].

Q15. Why does Java use a ListIterator instead of a plain Iterator for LinkedList in many cases?

A plain `Iterator` can only move forward and only supports `remove()`. A `ListIterator` can move both forward and backward (`hasNext/next` and `hasPrevious/previous`) and safely supports both `add()` and `remove()` during traversal — useful for a doubly linked structure that can be walked either direction.

Q16. What exception occurs if you remove an element from a LinkedList using a for-each loop directly, and why?

`ConcurrentModificationException` . The for-each loop uses an internal iterator that tracks a modification count; calling `list.remove(...)` directly (not through the iterator) changes that count outside the iterator's knowledge, so the next call to the iterator detects the mismatch and throws to prevent unpredictable behaviour.

Q17. Explain why size() is $O(1)$ even though the list has no backing array to measure.

LinkedList maintains a private `size` field that is incremented on every insertion and decremented on every removal, so `size()` just returns that stored value instead of walking the whole chain to count nodes.

Q18. Does LinkedList have a constructor that takes an initial capacity, like ArrayList's new ArrayList<>(50)? Why or why not?

No. Capacity hints only make sense when there is a backing array whose size can be pre-allocated. LinkedList has no backing array — every node is a separately allocated object — so there is nothing to pre-size.

Q19. What will this print, and why?

```
Deque<Integer> d = new LinkedList<>();
d.push(1);
d.push(2);
d.addLast(3);
System.out.println(d);
```

[2, 1, 3] . `push(1)` adds 1 to the front → [1]. `push(2)` adds 2 to the front → [2, 1]. `addLast(3)` adds 3 to the back → [2, 1, 3].

Q20. What is the time complexity of add(i, x) (inserting at a specific middle index) on a LinkedList, and why isn't it $O(1)$ even though linking is $O(1)$?

It is $O(n)$ overall: reaching index i requires walking i links from an end, which costs $O(n)$; only the actual re-linking once you're at the right node is $O(1)$. The search-to-find-the-spot dominates the total cost.

Q21. Write the missing line to make this a correct $O(1)$ enqueue-and-dequeue queue using LinkedList, and name the pattern.

```
Deque<Integer> queue = new LinkedList<>();
queue.offer(10);
queue.offer(20);
int first = ____; // should remove and return 10
```

`queue.poll()` . This is the FIFO (First-In-First-Out) queue pattern: elements are added at the back with `offer` and removed from the front with `poll` .

Q22. Why does a LinkedList generally use more memory per element than an ArrayList?

Each LinkedList node stores the value plus two object references (`next` and `prev`), plus normal Java object overhead. An ArrayList element is just a reference stored in a slot of a plain array, with no per-element next/prev overhead (though it may have some unused spare capacity).

Q23. True or False: Collections.sort() on a LinkedList sorts the nodes in place by swapping links.

False. Internally, the elements are first copied into a plain array, sorted there using TimSort ($O(n \log n)$), and then the LinkedList's nodes are relinked in that sorted order. Swapping links directly would be slower and much harder to implement correctly.

Q24. In a singly linked list you build by hand (only a next reference, no tail pointer), why is addLast(x) $O(n)$?

Without a tracked tail reference, the only way to find the last node is to start at `head` and follow `next` until reaching a node whose `next` is null — that walk visits every existing node, making it $O(n)$. Keeping a separate `tail` reference (as Java's real LinkedList does with `last`) would make it $O(1)$.

Q25. What does descendingIterator() do, and why is it possible on a LinkedList but awkward on a singly linked list?

It returns an iterator that walks the list from `last` to `first` . This works cleanly because every node has a `prev` reference to follow backward. On a singly linked list (only `next` references), there is no way to move backward from a node without re-walking from the head each time, making reverse traversal much more expensive.